```
#ifndef ARTERY_KALMANFILTERIND_H_
#define ARTERY_KALMANFILTERIND_H_

#include <vector>
#include <cmath>

/*
 * Implementasi Kalman Filter 2D (Constant Acceleration Model)
 * State Vector 6D: [x, y, vx, vy, ax, ay]
 */
class KalmanFilterIND {
public:
    KalmanFilterIND() {
        // Matriks transisi state (F) - 6x6
        F_ = std::vector<double>(36, 0.0);
        for(int i=0; i<6; i++) F_[i*6 + i] = 1.0; // Inisialisasi
Diagonal Utama (Identitas)
        // Nilai dt dan 0.5*dt^2 akan diisi di updateDt()

        // Matriks observasi (H) - 2x6 (Kita hanya mengobservasi x dan
y dari CAM)
        H_ = std::vector<double>(12, 0.0);
        H_[0] = 1.0;  // H(0,0) untuk x
        H_[7] = 1.0;  // H(1,1) untuk y

        // State vector [x, y, vx, vy, ax, ay]
        x_ = std::vector<double>(6, 0.0);

        // Kovarians estimasi (P) - 6x6
        P_ = std::vector<double>(36, 0.0);
        P_[0] = 1000.0;  P_[7] = 1000.0;   // Ketidakpastian posisi
awal
        P_[14] = 1000.0; P_[21] = 1000.0;  // Ketidakpastian kecepatan
awal
        P_[28] = 1000.0; P_[35] = 1000.0;  // Ketidakpastian
akselerasi awal

        // Noise proses (Q) - 6x6
        Q_ = std::vector<double>(36, 0.0);

        // Noise pengukuran (R) - 2x2
        R_ = std::vector<double>(4, 0.0);
        R_[0] = 0.1;
        R_[3] = 0.1;

        is_initialized_ = false;
        last_timestamp_ = 0.0;
```

```
    }

    void init(double x, double y, double timestamp) {
        x_[0] = x; x_[1] = y;
        x_[2] = 0; x_[3] = 0; // vx, vy awal
        x_[4] = 0; x_[5] = 0; // ax, ay awal
        last_timestamp_ = timestamp;
        is_initialized_ = true;
    }

    bool isInitialized() const {
        return is_initialized_;
    }

    void update(double x, double y, double timestamp) {
        if (!is_initialized_) {
            init(x, y, timestamp);
            return;
        }

        double dt = timestamp - last_timestamp_;
        last_timestamp_ = timestamp;
        if (dt <= 0) dt = 0.01;

        updateDt(dt);
        predict();

        std::vector<double> z = {x, y};
        std::vector<double> innovation = {z[0] - x_[0], z[1] - x_[1]};

        // S = H * P_ * H_transpose + R (H hanya mengekstrak 2x2 pojok
kiri atas dari P)
        std::vector<double> S(4, 0.0);
        S[0] = P_[0] + R_[0]; // P(0,0)
        S[1] = P_[1];         // P(0,1)
        S[2] = P_[6];         // P(1,0)
        S[3] = P_[7] + R_[3]; // P(1,1)

        double det_S = S[0] * S[3] - S[1] * S[2];
        if (std::abs(det_S) < 1e-9) det_S = 1e-9;
        std::vector<double> S_inv = {S[3] / det_S, -S[1] / det_S, -
S[2] / det_S, S[0] / det_S};

        // K = P_ * H_transpose * S_inv (K ukuran 6x2)
        std::vector<double> K(12, 0.0);
        for(int i = 0; i < 6; i++) {
            K[i*2 + 0] = P_[i*6 + 0] * S_inv[0] + P_[i*6 + 1] *
```

```
S_inv[2];
            K[i*2 + 1] = P_[i*6 + 0] * S_inv[1] + P_[i*6 + 1] *
S_inv[3];
        }

        // x = x + K * innovation
        for(int i = 0; i < 6; i++) {
            x_[i] += K[i*2 + 0] * innovation[0] + K[i*2 + 1] *
innovation[1];
        }

        // P = (I - KH) * P
        std::vector<double> I_KH(36, 0.0);
        for(int i = 0; i < 6; i++) {
            I_KH[i*6 + i] = 1.0;
            for(int j = 0; j < 6; j++) {
                double kh = 0.0;
                for(int k = 0; k < 2; k++) {
                    kh += K[i*2 + k] * H_[k*6 + j];
                }
                I_KH[i*6 + j] -= kh;
            }
        }

        std::vector<double> P_old = P_;
        std::vector<double> P_new(36, 0.0);
        for(int i = 0; i < 6; i++) {
            for(int j = 0; j < 6; j++) {
                for(int k = 0; k < 6; k++) {
                    P_new[i*6 + j] += I_KH[i*6 + k] * P_old[k*6 + j];
                }
            }
        }
        P_ = P_new;
    }

    std::pair<double, double> predict(double delta_time) {
        // Ekstrapolasi dengan Constant Acceleration
        double pred_x = x_[0] + x_[2] * delta_time + 0.5 * x_[4] *
delta_time * delta_time;
        double pred_y = x_[1] + x_[3] * delta_time + 0.5 * x_[5] *
delta_time * delta_time;
        return {pred_x, pred_y};
    }

private:
    std::vector<double> x_;
```

```
    std::vector<double> F_;
    std::vector<double> P_;
    std::vector<double> Q_;
    std::vector<double> H_;
    std::vector<double> R_;

    bool is_initialized_;
    double last_timestamp_;

    void predict() {
        std::vector<double> x_new(6, 0.0);
        for(int i = 0; i < 6; i++) {
            for(int j = 0; j < 6; j++) {
                x_new[i] += F_[i*6 + j] * x_[j];
            }
        }
        x_ = x_new;

        std::vector<double> P_temp(36, 0.0);
        for(int i = 0; i < 6; i++) {
            for(int j = 0; j < 6; j++) {
                for(int k = 0; k < 6; k++) {
                    P_temp[i*6 + j] += F_[i*6 + k] * P_[k*6 + j];
                }
            }
        }

        std::vector<double> P_new(36, 0.0);
        for(int i = 0; i < 6; i++) {
            for(int j = 0; j < 6; j++) {
                for(int k = 0; k < 6; k++) {
                    P_new[i*6 + j] += P_temp[i*6 + k] * F_[j*6 + k];
                }
                P_new[i*6 + j] += Q_[i*6 + j];
            }
        }
        P_ = P_new;
    }

    void updateDt(double dt) {
        // Update Matriks Transisi F untuk CA Model
        F_[2] = dt;         F_[3] = 0.0;        F_[4] = 0.5 * dt * dt;
// Baris x
        F_[8] = 0.0;        F_[9] = dt;         F_[11] = 0.5 * dt *
dt; // Baris y
        F_[16] = dt;        F_[17] = 0.0;       // Baris vx
        F_[22] = 0.0;       F_[23] = dt;        // Baris vy
```

```
        // Update Matriks Kovarians Proses Q
        double dt2 = dt * dt;
        double dt3 = dt2 * dt;
        double dt4 = dt3 * dt;

        double noise_ax = 2.0; // Variance untuk perubahan akselerasi
        double noise_ay = 2.0;

        // Pemetaan Q matriks 6x6
        Q_[0] = (dt4 / 4.0) * noise_ax;   Q_[2] = (dt3 / 2.0) *
noise_ax;   Q_[4] = (dt2 / 2.0) * noise_ax;
        Q_[7] = (dt4 / 4.0) * noise_ay;   Q_[9] = (dt3 / 2.0) *
noise_ay;   Q_[11] = (dt2 / 2.0) * noise_ay;
        Q_[12] = (dt3 / 2.0) * noise_ax;  Q_[14] = dt2 * noise_ax;
Q_[16] = dt * noise_ax;
        Q_[19] = (dt3 / 2.0) * noise_ay;  Q_[21] = dt2 * noise_ay;
Q_[23] = dt * noise_ay;
        Q_[24] = (dt2 / 2.0) * noise_ax;  Q_[26] = dt * noise_ax;
Q_[28] = 1.0 * noise_ax;
        Q_[31] = (dt2 / 2.0) * noise_ay;  Q_[33] = dt * noise_ay;
Q_[35] = 1.0 * noise_ay;
    }
};

#endif /* ARTERY_KALMANFILTERIND_H_ */
```